

PKU Captain

面向北京大学学生的桌面 AI 助手——项目作业报告

侯宇泽

石一

邱文晰

2500013225

2500013039

2400010732

@RizzoHou (组长)

@by-lastime

@Q-star17

信息科学技术学院 | 面向对象程序设计实践 | 2026 年 7 月

一句话简介： PKU Captain 是一款面向北大学生的 PyQt6 桌面应用，以**仪表盘（信息总站）**为主舞台，聚合课表、DDL、课程通知、树洞与教务更新；侧栏是一个**可见工具调用**的对话式 LLM agent，通过工具调用与文档库检索基于真实校园情境作答，并编排多步工作流。项目代码约 1.6 万行、62 个模块、450 项自动化测试，四条平行的 OOP 继承体系支撑全部扩展。

1 程序功能介绍

PKU Captain 解决的是北大学生「信息分散」的痛点：作业在教学网、通知在 Blackboard、办事流程在教务部、闲聊在树洞——每处都需独立登录、界面各异、信息易过期，而通用 LLM 又不了解你的课表与临近 DDL。本应用把这些权威数据源接入一个统一界面，左侧仪表盘一屏总览，右侧 agent 用自然语言应答并自动化。产品重心是仪表盘，对话 agent 作为侧栏承担多步编排与自然语言查询，二者共享同一套后端工具。

1.1 六项核心功能

- 统一仪表盘**——今日课表、近期 DDL、未读课程通知、树洞消息、教务更新一屏呈现。卡片级**独立刷新**（点某卡只刷该卡）、**并行刷新**（线程池扇出，启动耗时取决于最慢单卡而非累加）、**磁盘缓存秒开**（先用上次结果绘制，再静默比对签名只重绘变更卡），且各卡**错误隔离**，一卡失败不影响其余。
- 对话侧栏**——自然语言查询全部 PKU 数据；工具调用过程以 `InlineToolCall` 内联可见；回答**流式**逐字输出；**思维链**可选显示（默认关闭）；支持多会话历史、复制、上下文用量计量与 LaTeX 公式渲染。
- 工具集**——可被 agent 调用的 Tool 子类：pku3b（作业 / 通知 / 课表 / 身份）、文档库检索与视觉阅读、持久记忆、日历提醒、P-Lib 资料、教务部资源与更新、树洞检索。均带 JSON schema，在线才注册联网工具。
- 多步工作流**——Workflow 子类组合多次工具调用完成复杂任务（今日简报、周复盘、课程补课）；既可由仪表盘按钮触发，也被包装成 `WorkflowTool` 让 agent 在工具调用循环中直接发起。
- 文档库（取代嵌入式 RAG）**——`doc_search` 对切分后的 PDF 清单做词法排序检索；`doc_read` 用 `pdftoppm` 渲染 PDF 页并作为图像交给**视觉模型直接阅读**，省去「PDF→文本」的信息损耗中间层。

6. **记忆系统**——持久化个人偏好（食堂口味、课程、作息），每次响应**自然融入**系统提示；支持自然语言一句话入库，仪表盘还能调用 LLM 把整段话**抽取**成若干原子事实分别存储。

1.2 支撑性亮点

- **双模型角色，运行时可切换**：text（文本，默认 DeepSeek）与 visual（视觉，默认 Kimi）两个角色，端点 / 模型 / 密钥均可在设置中配置，任何 OpenAI 兼容端点均可接入；切换即重置对话并按视觉能力启停 doc_read。
- **统一账号中心**：一个「设置」入口，分「统一身份·树洞 / P-Lib / 模型配置 / 网络代理」四个标签页；一次 IAAA 登录同时供给树洞与 pku3b。
- **进程级网络代理**：跟随系统 / 直连 / 自定义三态，保存即对所有 requests 路径（含 vendored 库与 LLM）生效。
- **离线优先**：默认离线启动（EchoLLMProvider + 离线工具子集），无需密钥即可跑通全链路 with 全部测试；python -m src --online 切换在线。

1.3 工程规模一览

指标	数量	指标	数量
src/ 代码行数	≈16,000	OOP 继承体系	4
Python 模块	62	Vendored 校园客户端	4
自动化测试 (pytest)	≈450	可注册工具 (Tool)	13+

2 项目各模块与类设计细节

2.1 总体架构：两条泳道与一道接缝

系统是一个 PyQt6 桌面应用，划分为 **GUI 泳道** (src/ui/) 与 **后端泳道** (src/core、llm、tools、workflows、rag)，二者由**唯一的工厂接缝**连接：GUI 从不直接构造具体的 Provider / Tool / Source，只调用 src/core/bootstrap.py 中的工厂 (build_agent、build_source_registry 等)，拿到抽象类型后交互。该接缝的公共 API、线程模型、事件流与错误约定由 docs/integration_contract_zh.md（集成契约）唯一定义，是跨泳道协作的硬约束。build_agent(offline=True) 一键换成离线实现，让 GUI 无需密钥即可开发。

2.2 四条平行的 OOP 继承体系（课程核心）

OOP 是本课程的考核重点。项目的全部可扩展性都收敛为一个统一范式：「**子类化 + 向对应注册表注册**」，杜绝 ad-hoc 分发，模块保持**无导入副作用**（注册发生在调用点 bootstrap.py，而非 import 时），充分体现开闭原则。

体系	抽象基类	注册表	典型子类
Tool	Tool / ToolResult	ToolRegistry	Clock (离线示例)、Memory、DocBase、pku3b*、Plib、Dean*、Treehole、CalendarReminder
Workflow	Workflow	WorkflowRegistry	Hello (离线示例) + WorkflowTool 适配器
LLM	LLMProvider	工厂选择 (无注册表)	Echo (离线)、DeepSeek、Kimi
Source	Source	SourceRegistry	Static (离线)、Dean、Calendar

每条体系都遵循「一个离线参考子类 + 若干真实子类」的模式，使内核循环与测试无需 API key 或联网即可运行。要点：

- **Tool**: 子类声明 name / description / parameters_schema (JSON schema)，实现 invoke(args) → ToolResult。ToolResult 恒被填充 (成功或失败都有结构化结果)，并可携带 images——供视觉模型直接阅读的图像数据 (doc_read 渲染的 PDF 页即走这条路)。ToolRegistry.to_openai_schema() 把工具集序列化为 OpenAI 工具调用格式；unregister 用于按当前模型角色动态启停 doc_read。
- **LLMProvider**: 统一 chat() 与 stream_chat() 接口；ChatStreamEvent 分离答案增量与思维链增量两路 token 流。DeepSeek 实现真正的 SSE 流式与「思维链回放」 (推理模型要求把上一轮 reasoning_content 回传，否则 400)，Kimi 提供视觉能力。二者的 stream_chat 都按原始字节流再统一 UTF-8 解码，避免在分块边界截断多字节中文。
- **Workflow**: 构造时注入 ToolRegistry，在 run 中按名字组合工具调用；通过注册表成员检查，未注册 / 失败的工具有会被记录并跳过。agent_callable 标志决定是否被包成工具暴露给模型。
- **Source**: 知识源按各自 refresh_interval 轮询，fetch() 返回 Chunk 序列；采集 pipeline 对所有源统一处理。

2.3 Agent 内核与回合循环

Agent.turn() 是对话的心脏，最多迭代 8 轮：

```

user msg → Conversation.add_user → loop (≤8):
  取快照并把记忆折叠进领头 system 消息 (逐轮刷新，不写入 Conversation)
  → llm.stream_chat → 事件: reasoning_delta • assistant_delta • llm_response
  → 若有 tool_calls: tool_call → tool.invoke → tool_result (逐个)
    ↳ 若 ToolResult.images → 在所有结果后注入一条多模态 user 消息 (视觉模型读图)
  → context_usage → (无 tool_calls) → final

```

关键设计不变量：**记忆折叠进快照而非写入对话**——bootstrap 让 MemoryTool 与 Agent 共享同一个 MemoryStore 实例，每轮把当前条目经 render_memory_context() 合并进快照副本的领头 system 消息，因而 mid-turn 写入的记忆能即时反映，而渲染历史保持干净。工具错误串在工具边界 (redact.py) 凭据脱敏后才进入对话，注入 / 持有的密钥永不进入 LLM 请求或落盘的会话文件。

2.4 核心服务类与 GUI 线程模型

后端 core/ 还提供一组单一职责的服务类：Conversation (历史)、MemoryStore / MemoryLearnService (记忆与 LLM 抽取)、CredentialStore (secrets/ 的唯一写入者与状态源)、network.ProxyConfig /

apply_proxy (进程级代理)、SessionStore / SessionTitler (多会话持久化与自动命名)、DashboardCache (卡片原始载荷落盘)、VisionRouter (培养方案类问题自动切到视觉角色)。

GUI 侧遵循集成契约的三线程模型：AgentWorker (对话回合)、DashboardWorker (仪表盘刷新)、WorkflowWorker (工作流) 三个 QThread，外加一个 QThreadPool (tool_call_worker.run_async 承接对话框的阻塞调用)，保证窗口始终响应。取消回合用 threading.Event 而非 @pyqtSlot (worker 正阻塞在回合内)。

2.5 Vendored 在进程内驱动的校园客户端

pku3b / P-Lib / 树洞 / 教务部四个校园客户端均以 git subtree 形式 vendored 进 vendor/，映射为顶层包 (pypku3b / plib_cli / dean / treehole) 并在进程内直接驱动——没有子进程、没有兄弟 .venv、没有外部二进制，应用完全自包含。其中 pypku3b 用 requests+bs4 纯 Python 重写了原 Rust 版 pku3b 用到的子集 (教学网 Blackboard + 信息门户，走 IAAA SSO)，并把 TLS 固定到系统信任库 (certifi 无法验证 course.pku.edu.cn 的 GlobalSign 证书链)。

2.6 运行时数据流

除对话回合外，另有三条关键数据流，均遵循「后台线程 + 错误隔离 + 缓存优先」的一致设计：

- **仪表盘刷新**——DashboardWorker 用 ThreadPoolExecutor 把选中的工具扇出并发执行，as_completed 从工作线程逐卡发射结果，因而启动耗时取决于最慢单卡而非累加；_invoke 吞掉每个异常、逐卡独立上报，实现**错误隔离**。DashboardCache 把各卡原始载荷落盘，启动先据此绘制，再静默刷新、只对签名变更的卡重绘。刷新范围可控：单卡按钮只发 partial_refresh_requested，表头「刷新」才重载全部。
- **会话持久化**——SessionStore 在每回合结束 / 关闭时（且仅在首个真实回合后）写 data/sessions/；SessionTitler 用非思考模式的轻量模型异步命名；restore_conversation 重放时会**丢弃不完整的 tool-call 尾**，避免中断的回合在重开时永久 400。
- **代理控制**——账号中心「网络代理」保存 → apply_proxy 写进程环境变量 → 所有 requests 调用 (vendored 客户端与 LLM 一体) 随即生效，无需重启。

2.7 可验证性与自动化测试

约 450 项 pytest 覆盖记忆 / 会话 / 工具 / 仪表盘并发刷新 / 视觉注入 / 上下文计量等核心路径；GUI 流程通过一个**无第三方依赖**的 headless 框架驱动真实 worker 线程并对渲染态断言。凡改动 core/、llm/ 或工具调用线格式，还必须跑 scripts/smoke_deepseek.py 做一次真实 DeepSeek 往返冒烟。python -m src.cli 是与 GUI 同一 build_agent 循环的 REPL，充当接缝一致性探针。

3 小组成员分工情况

本项目采用「组长吸收整合开销、成员自由认领」的协作模式：周任务以待办清单列出，不预分负责人，任一成员认领未勾选条目、完成后追加 GitHub handle 作为 involver。三名成员的主要贡献如下。

成员	学号	GitHub	主要负责
侯宇泽	2500013225	@RizzoHou	组长；整体架构与集成契约、Agent 内核、LLM Provider、工具框架、vendored 客户端、跨任务整合与评审
石一	2500013039	@by-lastime	GUI 泳道主体：主窗口、对话面板、工具调用可视化、仪表盘、流式 / 思维链渲染、多会话交互
邱文晰	2400010732	@Q-star17	知识与工具层：文档库检索与视觉阅读、记忆系统、Source 采集、部分校园工具、pytest 测试体系

侯宇泽 (组长, @RizzoHou) ——负责项目的骨架与接缝：搭建四条 OOP 抽象基类与注册表、Agent 内核回合循环、build_agent 工厂与集成契约；实现 DeepSeek / Kimi 两个 Provider 的 SSE 流式与思维链回放；把 pku3b 等四个校园客户端 vendored 进仓库并改造为进程内驱动。此外承担全部跨任务整合、PR 评审协调与发布前的凭据审计，是团队的技术中枢。

石一 (@by-lastime) ——独立承担了 GUI 泳道的主体工程，工作量可观。他从零搭起 MainWindow 外壳与两栏 (仪表盘 | 对话) 布局，实现了对话面板的收发、Markdown / LaTeX 渲染、消息复制与「每个 assistant 片段一个气泡」的分段渲染逻辑；打磨了工具调用的内联可视化 (InlineToolCall)，让 agent 的每一次工具调用与结果对用户透明；完整实现了仪表盘的今日课表 / 近期 DDL / 通知等卡片与配套的 PKU3bCourseTableTool，并处理了流式输出、可选思维链窗口的自适应尺寸、多会话历史开关等一系列棘手的 Qt 细节。GUI 是用户对本产品的第一观感，石一的工作直接决定了应用「像不像一个成品」，其对渲染顺序、线程安全与视觉一致性的细致把控贯穿整个 UI 层。

邱文晰 (@Q-star17) ——负责知识、工具与质量保障层，覆盖面广且技术含量高。她实现了取代嵌入式 RAG 的文档库子系统——doc_search 词法检索与 doc_read 视觉阅读的完整链路，以及配套的 DocBaseReader 服务；构建了持久化记忆系统 (MemoryStore 的内容哈希入库、MemoryLearnService 的 LLM 事实抽取)，让个人偏好能自然融入每次响应；接入并调试了多个校园信息源与工具 (Source 采集 pipeline、教务部资源等)，并独立搭建了项目的 pytest 测试体系——目前 450 项测试覆盖记忆、会话、工具、仪表盘刷新并发、视觉注入等核心路径，是「main 永远可演示」的质量底座。测试与知识层往往是最容易被忽视却最耗心力的部分，邱文晰在这两块投入为项目稳定性提供了关键保障。

4 项目总结与反思

4.1 一次坦诚的承认

我们必须诚实：这个项目很庞大——约 1.6 万行代码、62 个模块、四条 OOP 继承体系、四个 vendored 校园客户端、450 项测试，覆盖流式 LLM、视觉多模态、并发刷新、进程级代理、凭据脱敏等诸多细节。以三名本科生在一门课的时间预算，这显然不是能够「手搓」出来的规模。因此我们不回避：本项目的大部分代码是「vibe」出来的——由 LLM 在我们的引导下生成。

但我们认为，真正有价值、也真正属于我们的贡献，不在于逐行敲出这些代码，而在于探索：在 agentic coding 的时代，如何 vibe 出高质量的代码。当写代码的边际成本趋近于零，工程的重心就从「如何把代码写出来」上移到「如何保证一堆自己没有逐字读过的代码是正确、可维护、可审计的」。这正是本项目 DEVCHANGELOG、VERIFICATION、TASTES 等制品以及我们为之编写的对应 skill 存在的原因。

4.2 agentic 代码工程的方法论

我们把「人的注意力」从代码本身抽离出来, 重新投放到几个正交的轴上, 每个轴由一份 stack-agnostic (与技术栈无关) 的散文制品承载, 并由一个项目级 skill 自动维护、按变更类型触发:

- **CLAUDE.md (活的不变量)** ——记录「绝不能破坏」的规则(如「记忆 store 必须共享」「累积而非替换」), 是每次会话载入的项目宪法。
- **ARCHITECTURE.html (结构)** ——盒箭图式的系统结构地图, 让不写代码的架构师也能一眼看清「谁和谁通信、边界在哪」。仅结构变更时更新。
- **DEVCHANGELOG.md (为什么)** ——决策日志, 记录「为何选 X 而非 Y」。它与记录「交付了什么」的 CHANGELOG.md 正交: 同一次改动可在两份文件里各留一条, 分处不同的轴。
- **VERIFICATION.md (怎么验)** ——把验证从「读代码」搬到「可观察行为»: 一份非程序员也能照着执行的、可复制粘贴的人工验证步骤清单, 作为 pytest 之外的验证地板, 专门覆盖 GUI 行为、真实网络往返、凭据处理等自动化测试难以覆盖之处。
- **TASTES/ (怎么写才好看)** ——规定性的编码品味指南, 从本仓库自身的教训中蒸馏(子类 + 注册、无导入副作用、累积而非替换、原始字节解码中文), 而非照搬网络上的通用说教。
- **集成契约 (接缝)** ——把 GUI 与后端之间的公共 API、线程模型、事件流、错误约定固化成硬约束, 破坏性变更必须显式标注, 让两条泳道可以并行演进而不互相蹂躏。

在流程上, 我们还引入了 **plan-gate**: 任何非平凡 / 结构性改动, 都必须先用散文陈述计划(涉及哪些文件、方案、可能破坏什么、有何假设), 让「架构师」评审计划而非 diff——把「要不要新增一个服务 / 全局」这类边界决策上提到计划层, 把「用哪个 Qt 信号、调哪个 API」这类惯用法下放到代码层。而「子类 + 注册」这一贯穿全项目的纪律, 本身就是一道结构护栏: 它把「新增能力」约束成一种可预测、可枚举、可测试的动作。

4.3 哲学: 从写代码到定契约

我们逐渐意识到, **vibe coding** 的反面不是「亲手写每一行」, 而是「放任」。一个由 LLM 生成的大型系统之所以可能失控, 不是因为代码由机器所写, 而是因为无人人为它维护一套人类可审计的真相——结构、决策、规则、品味、验证分别散落又互相污染。我们的应对, 是把这些真相显式化、正交化、并各自配一个自动维护它的 skill, 从而让一个不写代码的架构师也能审计整个系统的健康度。

这套方法也暴露了自身的边界, 我们同样如实记录: 例如凭据泄露对行为测试是不可见的(应用一边泄密一边正常工作), 只能靠发布前人工审计; 例如 LLM 生成的代码在局部正确却可能在系统级引入隐蔽的状态耦合(我们用「共享单例」「累积而非替换」等不变量反复堵这类洞)。承认边界, 本身就是方法论的一部分。

回头看, 这个项目与其说是「三个学生做了一个 App」, 不如说是一次关于 **agentic software engineering** 的方法论实验: 我们交付了一个能跑的产品, 但更想交付的, 是一套「在写代码几乎免费的时代, 如何让免费的代码依然可信」的工作方式。这些方法论制品与配套 skill, 是我们认为比代码本身更值得被审阅的部分。